A

PROTOTYPE BENCHMARK

AapatVani AI

What the prototype actually costs to run, measured rather than estimated: the road-access analysis, the model calls, the deployed service and the extraction accuracy — with the method for each so the numbers can be reproduced or disputed.

MEASURED
15 October 2026

DEPLOYED
Cloud Run · asia-south1

ENVIRONMENT
Node 25.2.1 · darwin/arm64

REPRODUCE
npm run bench · npm run eval

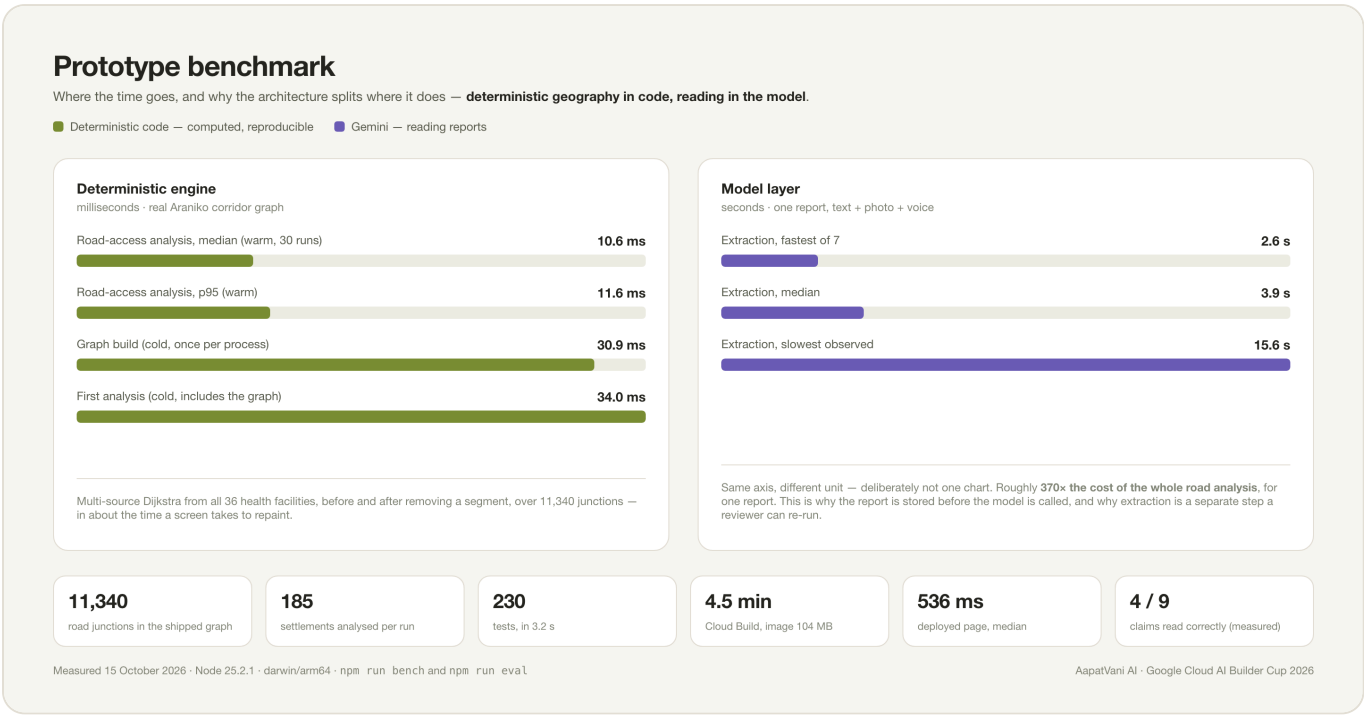
1 · What was measured, and how

Everything here was run against the real system — the shipped Araniko corridor dataset, the deployed Cloud Run service, and live model calls. No figure is an estimate. Each section names the command that produces it.

LAYER	METHOD	SAMPLE
Deterministic engine	`npm run bench`	30 warm runs of the full road-access analysis, plus cold measurements of the first call
Model layer	Live Gemini calls, recorded with each extraction	7 real extractions of a report carrying text, a photo and a voice note
Deployed service	HTTP requests against the Cloud Run URL	10 sequential page loads and two authenticated API reads
Build	Cloud Build, from source to a running revision	One full pipeline run
Accuracy	`npm run eval`, Gemini mode	One labelled scenario, 6 synthetic reports, against a hand-written answer key

> **The one-sentence result.** The road network analysis — which is the part nothing else in this space does — costs about **11 milliseconds**. Reading one report costs about **4 seconds**. That ratio is the architecture: everything that can be computed is computed, and the model is asked only to read.

2 · Results



Two panels, not one chart with two scales. Left, the deterministic engine in milliseconds; right, the model layer in seconds. Colour carries the same meaning it carries everywhere in the application: moss is what code computed, violet is what the model read.

Deterministic engine — `npm run bench`

MEASUREMENT	RESULT
Road-access analysis, median (warm, 30 runs)	**10.6 ms**
Road-access analysis, p95 (warm)	**11.6 ms**
Graph build (cold, once per process)	**30.9 ms**
First analysis (cold, includes the graph)	**34.0 ms**

Each run is a multi-source Dijkstra from all **36 health facilities**, twice — once on the intact network and once with a segment removed — over **11,340 junctions**, then a classification of every settlement. The whole thing finishes in about the time a screen takes to repaint, and because it is deterministic the result is identical on every run.

Model layer — recorded per extraction

MEASUREMENT	RESULT
Extraction, fastest of 7	**2.6 s**
Extraction, median	**3.9 s**
Extraction, slowest observed	**15.6 s**

Deployed service

MEASUREMENT	RESULT
Public page, median of 10 requests	**536 ms**
Public page, fastest / slowest	**286 ms / 4.04 s**
Authenticated API read	**330–709 ms**

The 4.04-second outlier is a cold start — the first request after the service has scaled to zero. Every other request is under a second. The service is configured with `min-instances` unset so that it idles free, which is a deliberate trade: a cold start for the first visitor, and no ongoing cost.

3 · Accuracy, measured

Performance is only half the picture; a fast wrong answer is worth nothing. The evaluation runs the real pipeline over labelled synthetic scenarios and compares the output with a hand-written answer key. On the demo scenario:

METRIC	RESULT
Claims read correctly	**4 / 9**
Hazards identified	**3 / 3**
Quoted excerpts verified against the original	**25 / 27**
Claims dropped for an invalid citation	**0**
Citations rejected by the server	**0 of 27**

> **4 of 9 is the honest headline, and it is the number we would point a judge at.** The deterministic half of the pipeline scores 100% on every metric — grouping, copy detection, contradictions, abstention and impact — but that is measured in a mode where the answer key is fed in as if extraction had produced it. It says nothing about reading Nepali. Four of nine is the first figure that describes the model, and it is lower, which is the point of publishing it.

The 25 of 27 excerpts is the other number worth reading. Every quotation a claim rests on is checked character-for-character against the reporter's own words, and two of them failed that check — which is the check working. Claims whose quotations cannot be found are not deleted; they are relabelled as interpretation and shown that way to a reviewer.

What the evaluation does not measure

- **One scenario of five.** The remaining four need more model quota than a metered key allows in a day, and a partial run is not reported as a whole one.
- **Real-world effectiveness.** This is a small, hand-written set over real road geography. It shows the pipeline behaves as designed on cases we chose.
- **A genuinely messy dataset.** Real forwarding is a screenshot, a re-recording, a translation chain. It has never met one.

4 · How to read these numbers

The split is the finding

A **370×** difference between the two layers is not an accident of implementation. It is the reason the architecture is shaped the way it is:

- **The report is stored before any model is called.** At 4 seconds a report, and with a model that can be rate-limited or unavailable, a submission must never wait on — or be lost to — a model call.
- **Extraction is a separate step a reviewer can re-run.** If it fails, the report is already safe and the reviewer reads it directly.
- **Geography never touches the model.** Coordinates come from GPS, a pin or a gazetteer match. The analysis is 11 milliseconds and identical every time, which is what makes it something a responder can act on.

What the numbers are not

- **Not a load test.** Every figure is single-digit concurrency on a demo deployment. There is no throughput or sustained-load measurement here.
- **Not a comparison.** Nothing is benchmarked against another product or model.
- **Not warm-cache-free.** The warm analysis figures reuse a graph already in memory; the cold figures are given separately for exactly that reason.
- **Not free of the network.** The model figures include round-trip time to the API, which is part of what a user waits for but is not the model's own compute.

Reproducing any of it

COMMAND	PRODUCES
`npm run bench`	The deterministic figures, on your machine, against the shipped dataset
`npm run eval`	Oracle mode — the deterministic half, no model needed
`npm run eval -- --mode gemini`	Real extraction accuracy; needs credentials and a fresh daily quota
`npm test`	230 tests in about 3 seconds

 Road data © OpenStreetMap contributors (ODbL 1.0). All benchmark figures measured 15 October 2026 on the configuration described in section 1.